Relatório de Estudo – Simulador de Gerenciamento de Memória (TP2)

Material para estudar o código e se preparar para explicá-lo na apresentação.

---

1. Visão geral

O programa lê um arquivo com comandos `IN`, `OUT` e `ACC` e simula como o sistema operacional distribui a memória principal entre processos, em três modos:

Modo	Classe	Arquivo	Ideia central
1	`ParticoesVariaveis`	`particoes.py`	cada processo recebe exatamente o que pediu (contíguo); política Worst-Fit ou Circular-Fit
2	`Buddy`	`buddy.py`	blocos sempre potência de 2, divididos ao meio e reunidos com o "irmão" (buddy)
3	`Paginacao`	`paginacao.py`	processo dividido em páginas de tamanho fixo, colocadas em frames quaisquer

O arquivo `simulador.py` é o "maestro": lê parâmetros, interpreta cada linha, chama a classe certa, imprime o estado e calcula o resumo final.

```
simulador.py --> main()
                 |- ler_parametros()  (argumentos ou perguntas)
                 |- criar_memoria()   --> ParticoesVariaveis | Buddy | Paginacao
                 '- Simulador.executar()
                       para cada linha:
                         interpretar_linha()  --> ("IN", id, tam) / ("OUT", id) / ("ACC",
id, end) / erro
                         memoria.entrar/sair/acessar()
                         memoria.estado()       (imprime)
                         atualizar_estatisticas()
                       imprimir_resumo()
```

Interface comum (polimorfismo por "duck typing")

As três classes têm **os mesmos métodos**, por isso o `Simulador` não precisa saber qual modo está rodando:

Método	Retorno	Significado
`entrar(pid, tam)`	`(sucesso, mensagem)`	comando IN
`sair(pid)`	`(sucesso, mensagem)`	comando OUT
`acessar(pid, end)`	`(sucesso, mensagem)`	comando ACC (modos 1 e 2 só avisam)
`estado()`	lista de strings	o que imprimir após cada linha
`ocupado()`	int	memória em uso (para "ocupação máxima")

Método	Retorno	Significado
`fragmentacao()`	int	modo 1: externa; modos 2/3: interna total

> **Pergunta provável:** "Por que não fizeram herança?" Resposta: em Python basta os objetos terem os mesmos métodos (duck typing). Uma classe base abstrata só acrescentaria código sem mudar o comportamento.

---

2. `simulador.py` – parte comum

2.1 Parâmetros

- **Sem argumentos**: `perguntar_parametros()` pergunta cada valor com `input()`.
- **Com argumentos**: `argparse` lê `--modo --memoria --politica --pagina --arquivo --passo`.
- `criar_memoria()` valida:
  - memória **potência de 2 em todos os modos** (o enunciado exige);
  - política `worst`/`wf` ou `circular`/`cf` no modo 1;
  - página potência de 2 **e** `<= memória` no modo 3.

Potência de 2 com bits: `n > 0 and (n & (n-1)) == 0`. Uma potência de 2 tem um único bit 1 (ex.: 32 = `100000`); subtraindo 1 todos os bits abaixo viram 1 (31 = `011111`), então o AND dá 0. Se houver outro bit ligado (48 = `110000`, 47 = `101111`), o AND não é 0.

2.2 Interpretação da linha (`interpretar_linha`)

Expressão regular: `NOME ( ID [, NUMERO] )` com espaços opcionais em qualquer lugar.

- `IN` exige número inteiro **> 0** (zero ou negativo = erro de comando).
- `OUT` exige **apenas** o ID.
- `ACC` exige número inteiro (negativo é aceito no parser e a memória responde "ENDEREÇO INVÁLIDO").
- Qualquer outra coisa → `ERRO DE COMANDO: ...` e a simulação **continua**.
- Linhas vazias e linhas começando com `#` são ignoradas.
- O ID é qualquer texto sem espaço/vírgula/parênteses (o PDF fala "inteiro positivo", mas os exemplos usam letras – aceitamos os dois).

2.3 Laço principal (`Simulador.executar`)

Para cada linha: executa, imprime `Linha N`, a mensagem (`OK: ...` ou erro) e o `estado()`, atualiza estatísticas e, se `--passo`, espera ENTER.

2.4 Estatísticas do resumo

Item	Como é calculado
Admitidos	`IN` que retornou sucesso

Item	Como é calculado
Rejeitados	`IN` que retornou exatamente `ESPAÇO INSUFICIENTE DE MEMÓRIA`
Ocupação máxima	maior `memoria.ocupado()` observado após cada linha (valor e %)
Fragmentação média	média de `memoria.fragmentacao()` após cada linha processada

`ocupado()` em cada modo:

- modo 1: soma das partições;
- modo 2: soma dos **blocos** (inclui a fragmentação interna);
- modo 3: frames ocupados × tamanho da página.

`fragmentacao()` em cada modo:

- modo 1: **externa** = total livre − maior bloco livre (memória livre que não pode ser usada por um único pedido contíguo);
- modos 2 e 3: **interna total**.

---

3. Modo 1 – Partições variáveis (`particoes.py`)

3.1 Estrutura de dados

Uma lista `blocos` em ordem de endereço que **cobre a memória inteira**: `[inicio, tamanho, dono]`, com `dono = None` para bloco livre.

Exemplo: `[[0,8,'A'], [8,4,'B'], [12,20,None]]`.

Vantagem: o mapa da memória é a própria lista e os vizinhos de um bloco são `i-1` e `i+1`.

3.2 IN (`entrar`)

1. Se o ID já está na lista → `ERRO: PROCESSO JÁ EXISTE` (checado **antes** do espaço).
2. Escolhe o índice do bloco conforme a política (abaixo). `-1` → `ESPAÇO INSUFICIENTE DE MEMÓRIA`.
3. Divide: o bloco vira `[inicio, tam, pid]` e, se sobrar, insere `[inicio+tam, sobra, None]` logo depois.
4. Circular-Fit: `ponteiro = inicio + tam`; se for igual ao tamanho da memória, vira 0.

Worst-Fit (`_escolher_worst_fit`) – percorre os livres em ordem de endereço e guarda o de maior tamanho usando `>` (estritamente maior). Assim, **no empate fica o primeiro (menor endereço)**.

Circular-Fit (`_escolher_circular_fit`) – monta a ordem de visita: `[livres com início >= ponteiro] + [livres com início < ponteiro]` e pega o primeiro que cabe. Rejeição **não** mexe no ponteiro; OUT também não.

> **Pegadinha:** se o ponteiro cair no meio de um bloco livre (depois de uma fusão), esse bloco tem

> início < ponteiro e só é visitado na "volta". Seguimos o texto do enunciado ao pé da letra.

3.3 OUT (`sair`)

1. Acha o bloco do processo (senão `ERRO: PROCESSO NÃO ENCONTRADO`).
2. Marca como livre.
3. Se o da **direita** é livre: soma e remove o da direita.
4. Se o da **esquerda** é livre: o da esquerda absorve o atual, que é removido.

Por que primeiro a direita? Porque ao remover `i+1` o índice `i` continua válido; depois a fusão com `i-1` remove `i`. Se fizesse ao contrário, o índice do bloco mudaria.

A memória **não é circular**: o último bloco nunca é unido ao primeiro (só olhamos `i-1` e `i+1` dentro dos limites da lista).

3.4 Exemplo do enunciado (memória 32)

Sequência: `IN A8, B4, C6, D5; OUT A; OUT C; IN E3; IN F6; IN G7; OUT B`

Depois de `OUT A` e `OUT C`, os livres são `[0..7](8)`, `[12..17](6)`, `[23..31](9)`.

Passo	Worst-Fit	Circular-Fit (ponteiro antes → depois)
IN E3	maior = 9 em 23 → E em 23..25	ptr 23: bloco 23 cabe → E 23..25 (ptr 26)
IN F6	maior = 8 em 0 → F em 0..5	ptr 26: bloco 26 (6) cabe → F 26..31 (ptr 32 → **0**)
IN G7	maiores = 6 → **rejeitado**	ptr 0: bloco 0 (8) cabe → G 0..6 (ptr 7)
OUT B	6..7 + 8..11 + 12..17 = **12**; livres **12 | 6**	7 + 8..11 + 12..17 = **11**

Mesma sequência, resultados diferentes: o Worst-Fit "gastou" o bloco de 8 com F e não sobrou espaço contíguo para G.

---

4. Modo 2 – Buddy (`buddy.py`)

4.1 Estruturas

- `livres`: lista de `(endereco, tamanho)`.
- `alocados`: `pid -> (endereco, tamanho_do_bloco, tamanho_pedido)`.

4.2 IN

1. `k = proxima_potencia_de_2(tam)` (dobra a partir de 1 até `>= tam`).
2. Candidatos = livres com tamanho `>= k`. Nenhum → rejeitado.
3. Escolhe o **menor** tamanho; empate → **menor endereço** (`min` com chave `(tamanho, endereco)`).
4. Enquanto o bloco for maior que `k`: divide ao meio, a metade **direita** `(end + t/2, t/2)` vai para os livres e continua com a **esquerda**.

5. Fragmentação interna do processo = `k - tam`.

4.3 OUT – coalescência

```
enquanto t < memória:
    buddy = endereco XOR t
    se (buddy, t) não está livre: para
    remove buddy; endereco = min(endereco, buddy); t = 2t
adiciona (endereco, t) aos livres
```

Por que XOR? Os blocos de tamanho `t` estão alinhados em múltiplos de `t`. Dois buddies diferem só no bit que vale `t`: o da esquerda tem esse bit 0 e o da direita tem 1. O XOR inverte esse bit. Ex.: bloco 8 em 8 → buddy = 8 XOR 8 = 0; bloco 16 em 0 → buddy = 0 XOR 16 = 16.

> **Pegadinha:** dois blocos livres vizinhos de mesmo tamanho **que não são buddies** (ex.: 8@8 e 8@16) não podem ser unidos, pois 8 XOR 8 = 0 ≠ 16. A condição `(buddy, t) in livres` também exige o **mesmo tamanho**: se o buddy foi dividido, ele não aparece inteiro na lista e não há fusão.

4.4 Exemplo do enunciado (memória 64)

Comando	Bloco	Frag. do processo	Frag. total
IN A5	64→32→16→8, A em 0..7	3	3
IN B9	menor livre ≥16 é 16@16 → B 16..31	7	10
IN C20	32@32 → C 32..63	12	22
IN D6	8@8 → D 8..15	2	24
IN E3	nada livre → **rejeitado**	–	24
OUT A	buddy 8@8 ocupado (D) → livre 0..7		21
OUT D	buddy 0@8 livre → 16@0; buddy 16@16 ocupado (B)		19
OUT B	16@16 + 16@0 → 32@0; buddy 32@32 ocupado (C)		12
OUT C	32@32 + 32@0 → 64@0		0

---

5. Modo 3 – Paginação (`paginacao.py`)

5.1 Estruturas

- `frames`: lista com `memória / página` posições; `None` (livre) ou `(pid, página)`.
- `processos`: `pid -> {"tam": tamanho, "tabela": [frame da página 0, 1, ...]}` (a tabela de páginas).
- `ultimo_pid`: processo do último comando, para imprimir a tabela dele.

5.2 IN

- `n = (tam + página - 1) // página` → é o **teto** de `tam / página` usando só inteiros.
- Frames livres em ordem crescente; se houver menos que `n` → rejeitado.
- Usa os `n` primeiros (menores índices). **Não precisam ser contíguos**.
- Frag. interna do processo = `n × página - tam` (sobra só na última página).

5.3 OUT

Libera todos os frames da tabela.

5.4 ACC – tradução de endereço

Ordem das verificações (importante):

1. Processo existe? Senão → `ERRO: PROCESSO NÃO ENCONTRADO`.
2. `0 <= endereço < tam`? Senão → `ERRO: ENDEREÇO INVÁLIDO`.
3. `página = end // tam_pag`, `desloc = end % tam_pag`, `frame = tabela[página]`, `físico = frame × tam_pag + desloc`.

Observação: o limite é o **tamanho do processo**, não o fim da última página. No exemplo, A tem 20 bytes em 3 páginas (24 posições), mas `ACC(A,22)` é inválido.

5.5 Exemplo do enunciado (memória 64, página 8 → 8 frames)

Comando	Resultado	Frag. total
IN A20	3 páginas → frames 0,1,2	4
IN B9	2 páginas → frames 3,4	11
IN C30	4 páginas, só 3 livres → **rejeitado**	11
IN D16	frames 5,6	11
ACC(B,8)	pág 1, desloc 0, frame 4 → **32**	11
ACC(A,22)	22 ≥ 20 → **ENDEREÇO INVÁLIDO**	11
OUT A	livres 0,1,2,7	7
IN C30	frames **0,1,2,7** (não contíguos!)	9

Note que `C` foi rejeitado e depois admitido: um processo rejeitado **nunca existiu**, então não há "PROCESSO JÁ EXISTE".

---

6. Decisões de projeto (o enunciado não define)

Situação	Decisão
ID com letras	aceito (exemplos usam letras)
`IN` com tamanho ≤ 0, comando desconhecido, falta de argumento	`ERRO DE COMANDO`, segue a simulação
`IN` maior que a memória	`ESPAÇO INSUFICIENTE DE MEMÓRIA` (conta como rejeição)

Situação	Decisão
Empate no buddy	menor endereço
`ACC` com endereço negativo	`ENDEREÇO INVÁLIDO`
`ACC` nos modos 1 e 2	aviso, nada muda
Fragmentação no modo 1	externa = livre − maior bloco livre
Média da fragmentação	média sobre todas as linhas de comando processadas
Linhas `#...`	comentários ignorados (úteis nos testes)

---

7. Testes

- `test_simulador.py` (`python3 -m unittest test_simulador.py -v`): confere automaticamente os três exemplos do PDF e regras de erro/parser.
- `testes/exemplo_modo1..3.txt`: exemplos do enunciado.
- `testes/teste_particoes.txt`, `teste_buddy.txt`, `teste_paginacao.txt` (25+ linhas cada): rejeições, `JÁ EXISTE`, `NÃO ENCONTRADO`, `ENDEREÇO INVÁLIDO`, `ACC` ignorado, sintaxe inválida, tamanho zero/negativo, comando desconhecido.
- `testes/carga_*.txt`: cargas maiores usadas no relatório de experimentos.

---

8. Perguntas prováveis na apresentação

1. **Qual a diferença entre fragmentação interna e externa?** Interna: espaço desperdiçado *dentro* do que foi dado ao processo (buddy arredonda, página sobra). Externa: espaço livre *fora* dos processos, mas picado em pedaços que não servem (partições variáveis).
2. **Por que partições variáveis não têm fragmentação interna?** Porque cada partição tem exatamente o tamanho pedido.
3. **Por que a paginação não tem fragmentação externa?** Qualquer frame livre serve para qualquer página; não é preciso contiguidade.
4. **Como funciona o ponteiro do Circular-Fit?** Fica no fim da última alocação, volta a 0 se chegar ao fim da memória, não muda em rejeição/OUT; a busca começa no primeiro bloco livre com início ≥ ponteiro e dá a volta.
5. **Por que Worst-Fit?** Deixar a maior sobra possível, para que ela ainda sirva a outro processo. Desvantagem: destrói os blocos grandes (no exemplo, G foi rejeitado).
6. **Como acha o buddy?** `endereço XOR tamanho`.
7. **Como calcula o número de páginas sem float?** `(tam + pag - 1) // pag`.
8. **Ordem dos erros no ACC?** Primeiro processo inexistente, depois endereço inválido.
9. **Por que no OUT funde primeiro com a direita?** Para não invalidar o índice `i` do bloco atual.
10. **O que conta como "ocupação"?** Memória reservada: no buddy e na paginação inclui a parte desperdiçada (bloco/frames inteiros), porque ela não pode ser usada por outro processo.
11. **Complexidade?** Cada operação percorre a lista de blocos/frames: O(n) por comando, suficiente para o tamanho das simulações.
12. **Por que `max(..., default=0)`?** Se não houver bloco livre, `max` de lista vazia daria erro.

---

9. Limitações conhecidas

- No modo 3, em uma linha com erro de sintaxe, a tabela exibida é a do último processo válido envolvido (a linha não tem ID reconhecível).
- O mapa é textual (lista de faixas), não gráfico.
- A ocupação/fragmentação é medida **após** cada linha, não em instantes intermediários.